

CSC 480: Artificial Intelligence

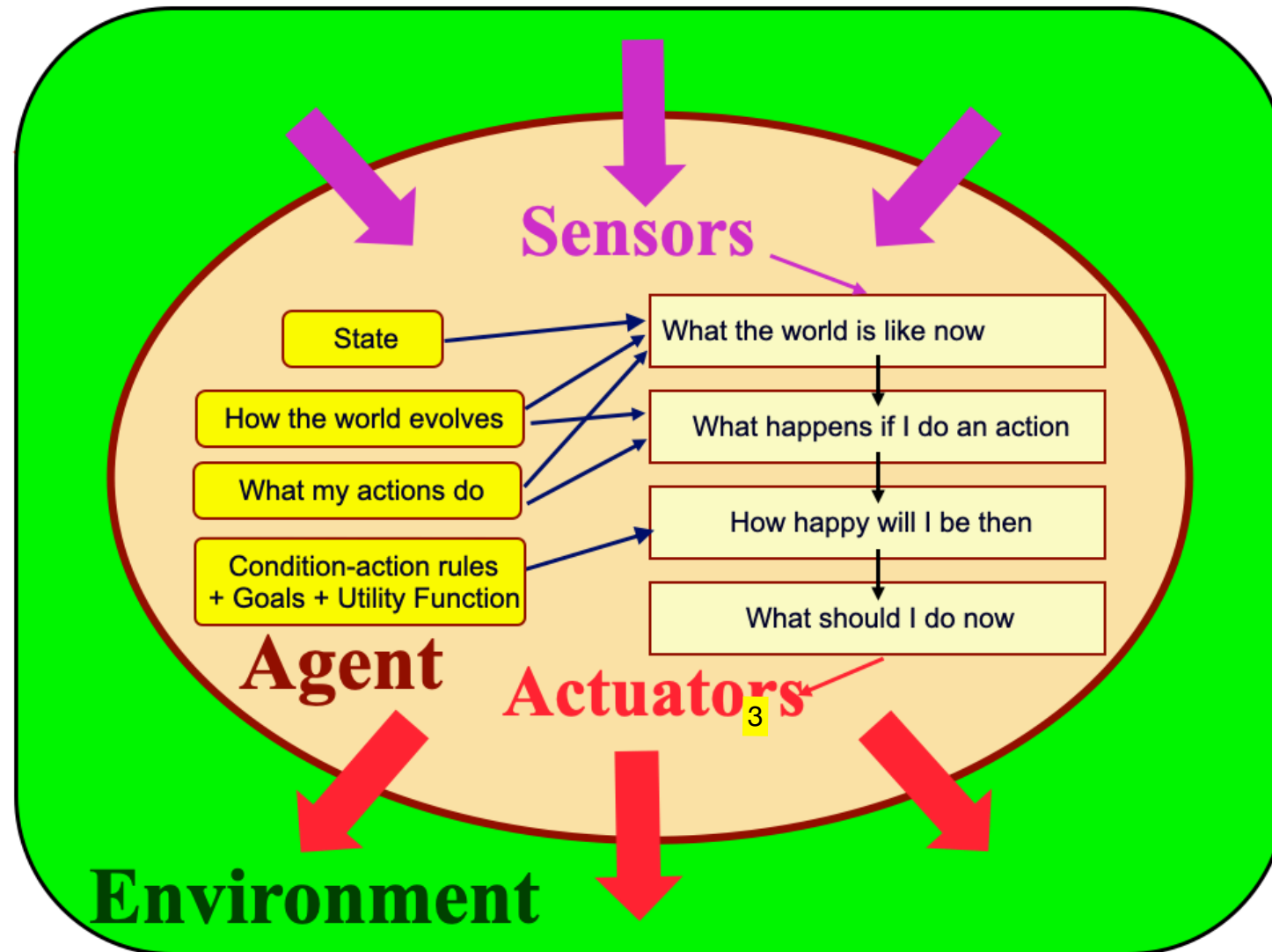
3- Tree Search - Uninformed Search

Rodrigo Canaan
Assistant Professor
Computer Science Department
Cal Poly, San Luis Obispo
rcanaan@calpoly.edu

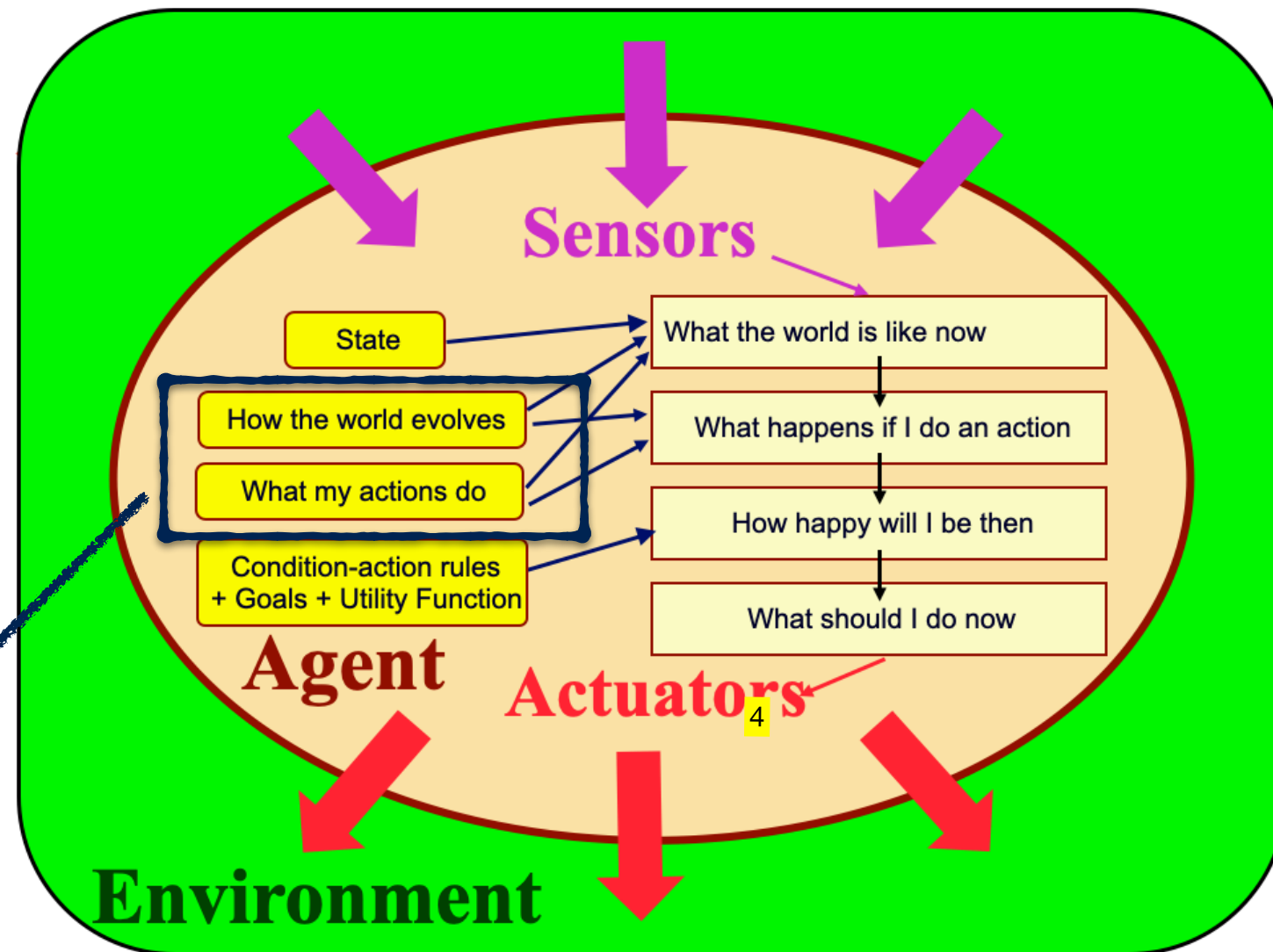
Jan 30, 2026

Tree Search

Recap: Utility-based agents



Recap: Utility-based agents



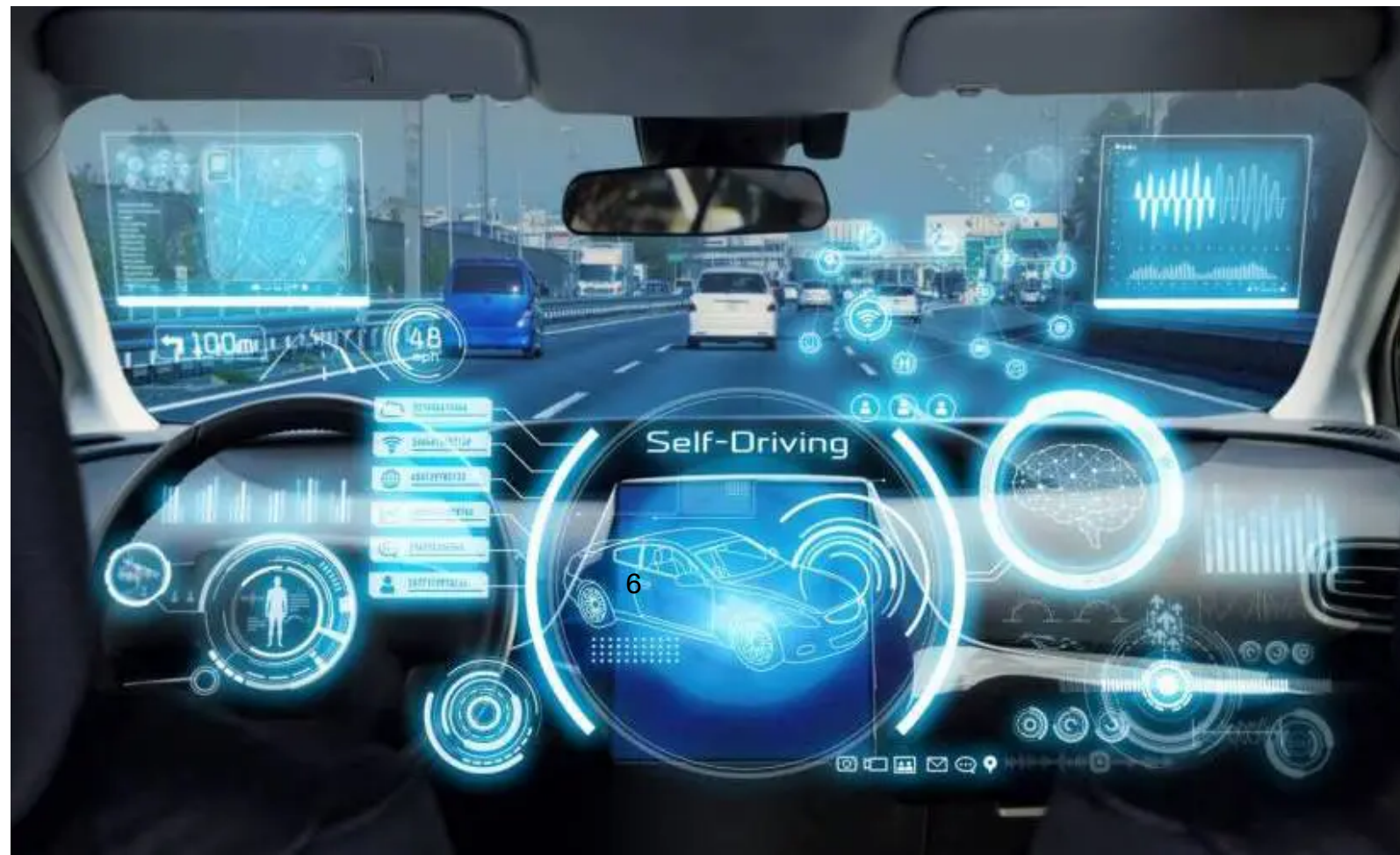
Forward model

Search

- **Key Insight:** if we have (or can build) a forward model of the environment, we can search for sequences of actions that lead to a goal state (or a state with high utility)
- Forward model: a function that takes a state and an action and (assuming environment is deterministic) outputs the next state
- For this to work, you must be able to simulate states with the forward model **much** faster than your observation-action loop

States

- **State:** representation of the current configuration of the environment
- Can be quite complex!

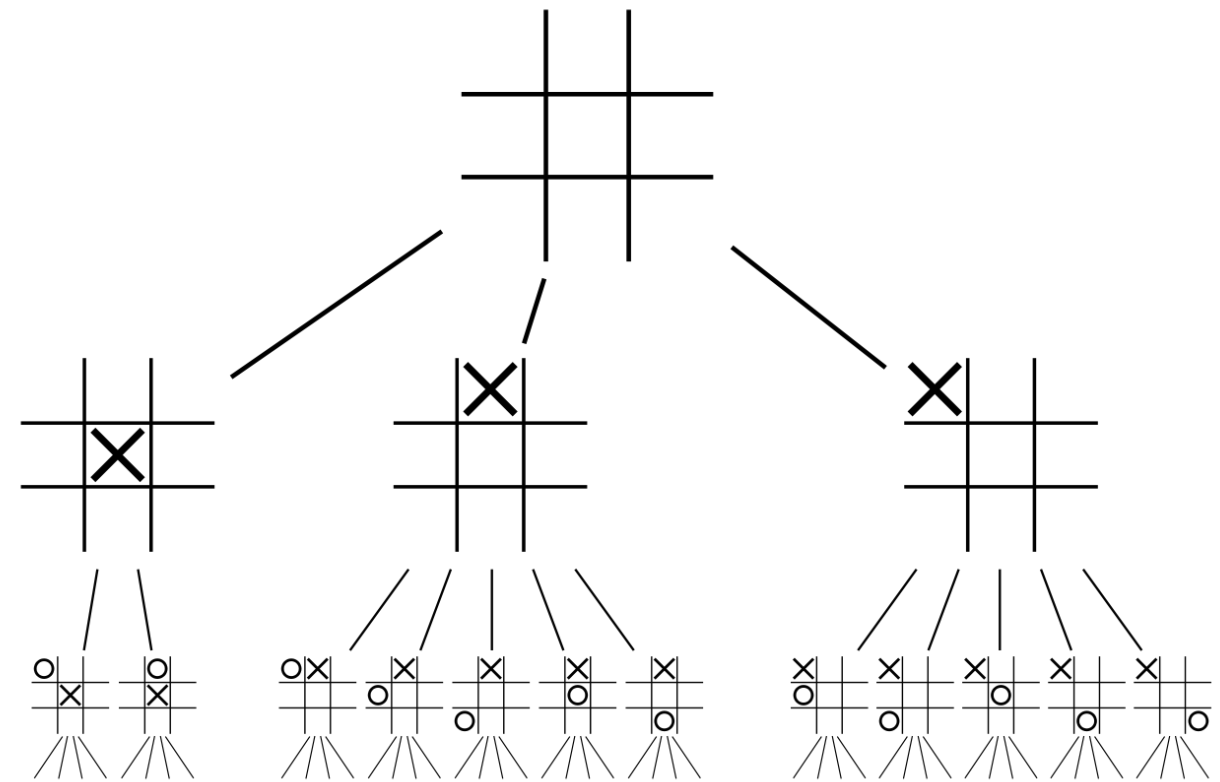


Simplifying assumptions

If we can...

- Represent states as discrete “black box” entities (**atomic representation**) and..
- quickly compute, given a state, which actions lead to which states (**forward model**)

Then searching for a solution becomes similar to finding a path on a map (or graph)



Problem Formulation

- What's the starting state?
- How are states represented?
- What actions are available at each state?
- Which state does each action at a given state take to?
(forward model)
- Which states are terminal?
- What's the reward at each terminal state (or at each transition)?

Representation of states

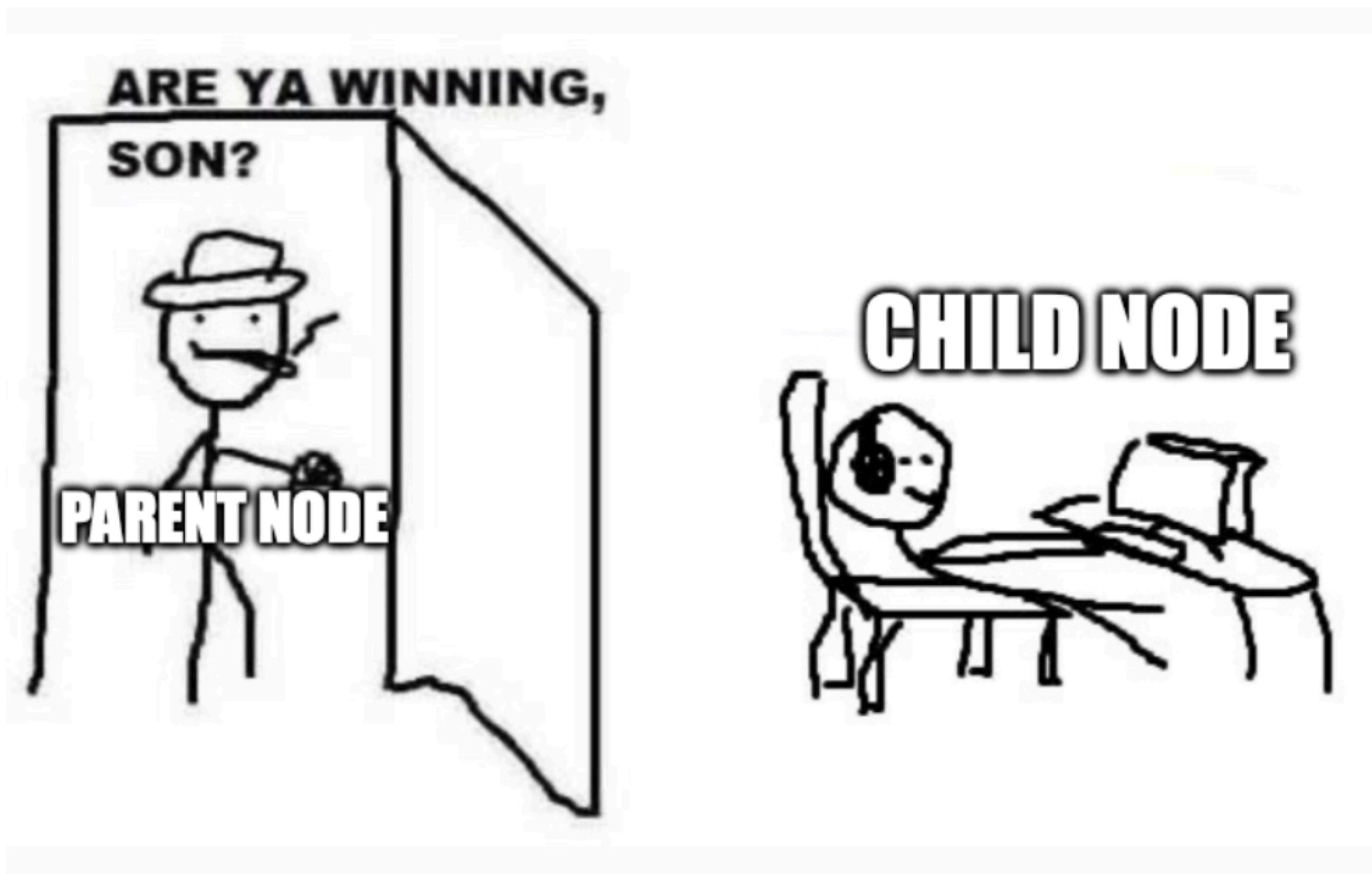
Representation	Description	What can we do with a state?	Example (Route Finding)
Atomic	Indivisible (black box) states	Check for terminal states, parent/children of a state, equality between states (maybe)	Name of the city/road we are in
Factored	States represented by arrays of properties	Check for value of each property, do math/logic with it	GPS coordinates, fuel levels
Structured	States represented by complex hierarchy of objects with their own properties	Check for objects within a state, their relationships and properties	Passengers, each with their names, luggage weight, hunger level, favorite food...

Representation of states

Representation	Description	What can we do with a state?	Example (Route Finding)
Atomic	Indivisible states	Check for terminal states, parent/children of a state, equality between states (maybe)	Name of the city/road we are in
Factored	States represented by arrays of properties	Check for value of each property, do math/logic with it	GPS coordinates, fuel levels
Structured	States represented by complex hierarchy of objects with their own properties	Check for objects within a state, their relationships and properties	Passengers, each with their names, luggage weight, hunger level, favorite food...

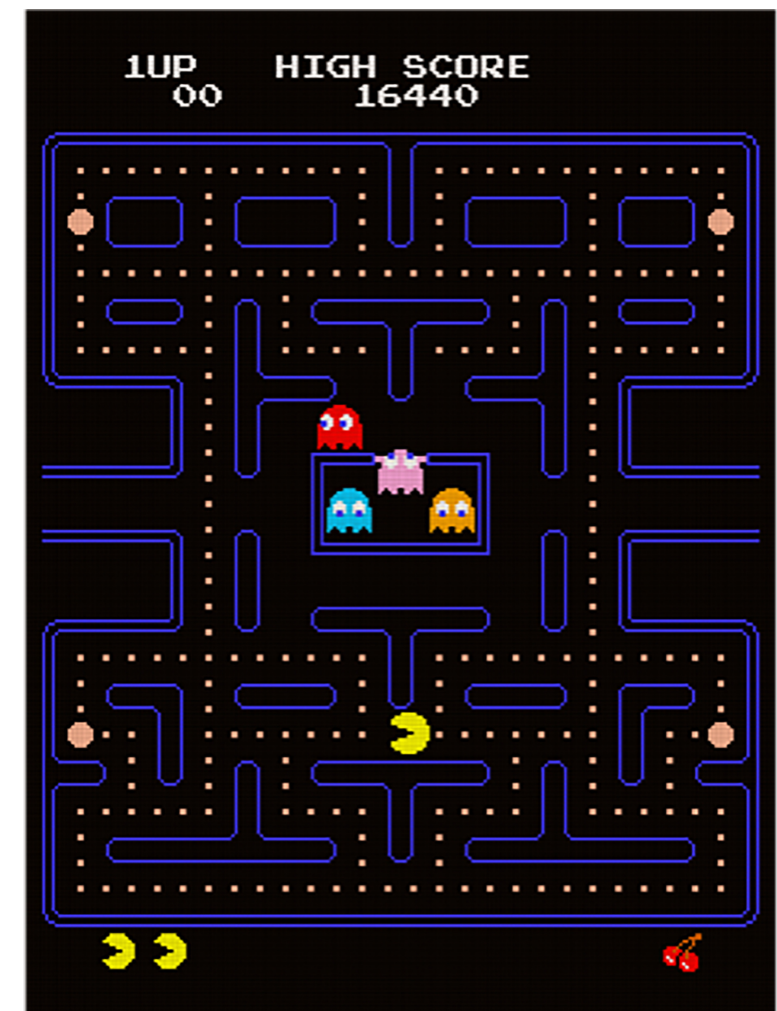
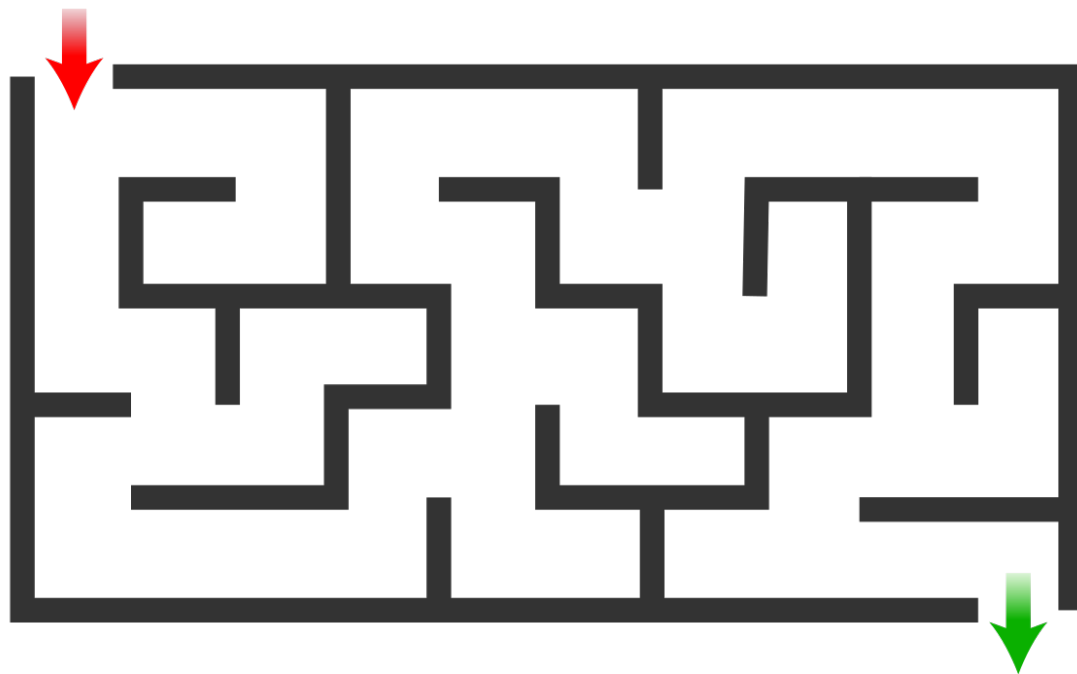
Recap: Atomic State

- Simplifying assumption: all you can ask of a node is
 - Who is its parent?
 - Who are its children?
 - Is the state a win?



Search in “physical” space versus game-state space

Question: If you move one step right, then left, are you in the same state?



Classical tree search

- Atomic representation of states
- Typical assumptions:
 - ➔ No randomness
 - ➔ No hidden information
 - ➔ Single-agent

Informed vs Uninformed Search

- Uninformed: no domain knowledge
 - All we can do is ask whether a state is terminal and what its reward is.
- Informed: some domain knowledge
 - A domain-specific *heuristic function* estimates the desirability of each state (e.g. expected score at the end of the game, expected cost to nearest win state).

States vs Nodes

- **State**: a representation of the environment at a certain point in time
 - Changes only when we choose an action
- **Node**: a data structure used to build the tree that represents our search for a solution
 - Usually contains a state and other metadata (pointer to parent/children, number of visits, heuristics).
 - While searching for an action, we may visit many nodes, but we don't move to a different state until we commit to an action.
 - For this to be possible, you must be able to simulate sequences of states **without** actually changing the environment
 - This allows you to backtrack!

Cost vs Complexity

- Do not confuse:
 - ➔ **Cost of a solution (path cost):** usually, # of nodes in the path (but edges may have different weights)
 - ➔ **Computational complexity of the search (search cost):** usually, proportional to total # of nodes visited (even ones that were not in the path to the solution)

Completeness and Optimality

- **Complete:** if a solution exists (with finite depth), the algorithm will find it
- **Optimal:** if multiple solutions exist, the algorithm will return one with minimal depth (or path cost)

Example Environments

Route Finding

- ❖ **states**

- ❖ locations

- ❖ **initial state**

- ❖ starting point

- ❖ **successor function (actions)**

- ❖ move from one location to another
 - ❖ it is common to simplify the space to a discrete sets of points, which leads to search in a graph
 - ❖ Each location is a vertex
 - ❖ If you can move from locations A to B, there is an edge between the corresponding vertices

- ❖ **goal test**

- ❖ arrive at a certain location

- ❖ **path cost**

- ❖ may be quite complex
 - ❖ money, time, travel comfort, scenery, ...

- ❖ **Important: All other problems in this section can be thought of as finding a route in a suitably-defined graph!**

n-queens

- ❖ Put n queens on a $n \times n$ board with no two queens on the same row, column, or diagonal
- ❖ see also [Wikipedia Eight Queens Puzzle](#)



8-Queens

Incremental Approach

- ❖ **start with an empty board**
- ❖ **add queens one by one**
 - ❖ no violation of constraints
 - ❖ different row, column, diagonal from sitting queens
- ❖ **incremental formulation**
 - ❖ states
 - ❖ arrangement of up to 8 queens on the board
 - ❖ initial state
 - ❖ empty board
 - ❖ successor function (actions)
 - ❖ add a queen to any square
 - ❖ goal test
 - ❖ all queens on board
 - ❖ no queen attacked
 - ❖ path cost
 - ❖ irrelevant (all solutions equally valid)
- ❖ **Properties: 3×10^{14} possible sequences; can be reduced to 2,057**

8-Queens

Complete-State Approach

- ❖ **start with a full board**
 - ❖ all n queens placed on the board
 - ❖ conflicts are to be expected
- ❖ **try to find a better configuration**
 - ❖ reduced number of conflicts
- ❖ **complete-state formulation**
 - ❖ states
 - ❖ arrangement of 8 queens on the board
 - ❖ initial state
 - ❖ all 8 queens on board
 - ❖ successor function (actions)
 - ❖ move a queen to a different square
 - ❖ goal test
 - ❖ no queen attacked
 - ❖ path cost
 - ❖ irrelevant (all solutions equally valid)
- ❖ **Properties: good strategies can reduce the number of possible sequences considerably**

8-Queens Refined

- ❖ **simple solutions may lead to very high search costs**
 - ❖ 64 fields, 8 queens ==> 648 possible sequences
- ❖ **more refined solutions trim the search space, but may introduce other constraints**
 - ❖ place queens on “unattacked” places
 - ❖ much more efficient
 - ❖ may not lead to a solution depending on the initial moves
 - ❖ move an attacked queen to another square in the same column, if possible to an “unattacked” square
 - ❖ much more efficient

Sudoku

❖ states

- ❖ Grid puzzle with cells filed with numbers 0-9 or a blank

❖ initial state

- ❖ A few cells initially filled

❖ successor function (actions)

- ❖ Add a number to a blank cell

❖ goal test

- ❖ All cell filled
- ❖ No cell sharing a row, column or 3x3 square have the same number

❖ path cost

- ❖ all solutions are equally valid

					1	2	3	
1	2	3			8		4	
8		4			7	6	5	
7	6	5						
						1	2	3
	1	2	3			8		4
	8		4			7	6	5
	7	6	5					

6	5	7	9	4	1	2	3	8
1	2	3	6	5	8	9	4	7
8	9	4	2	3	7	6	5	1
7	6	5	1	2	3	4	8	9
2	3	1	8	9	4	5	7	6
9	4	8	7	6	5	1	2	3
5	1	2	3	7	6	8	9	4
3	8	9	4	1	2	7	6	5
4	7	6	5	8	9	3	1	2

Bridge and Torch problem

- ❖ states
 - ❖ Location of four adventurers (A,B,C,D) and a torch (*) on either bank of a river crossed by a bridge
 - ❖ Example (A,B,* | C,D)
- ❖ initial state
 - ❖ All adventurers and the torch on the left bank of the river (A,B,C,D,* |)
- ❖ successor function (actions)
 - ❖ Move one or to two adventurers and the torch from the bank with the torch to the opposite bank
- ❖ Cost of an action
 - ❖ Each adventure takes a number of seconds to cross (1,2,5,8)
 - ❖ If two adventurers cross simultaneously, use the cost of the slower adventurer
- ❖ goal test
 - ❖ All adventurers and the torch on the right bank of the river (| A,B,C,D,*)
- ❖ path cost
 - ❖ Total cost of all crossing
- ❖ Variations exist, such as as “goats and cabbage”, “wolves and sheep” etc.
- ❖ In general, can be called “river-crossing problems”

Robot Navigation

- ❖ **states**

- ❖ locations
- ❖ position of actuators

- ❖ **initial state**

- ❖ start position (dependent on the task)

- ❖ **successor function (actions)**

- ❖ movement, actions of actuators

- ❖ **goal test**

- ❖ task-dependent

- ❖ **path cost**

- ❖ may be very complex
 - ❖ distance, energy consumption

Assembly Sequencing

- ❖ **states**
 - ❖ location of components
- ❖ **initial state**
 - ❖ no components assembled
- ❖ **successor function (actions)**
 - ❖ place component
- ❖ **goal test**
 - ❖ system fully assembled
- ❖ **path cost**
 - ❖ number of moves

Example: robotic assembly

- ❖ **states:**

- ❖ real-valued coordinates of robot joint angles parts of the object to be assembled

- ❖ **actions:**

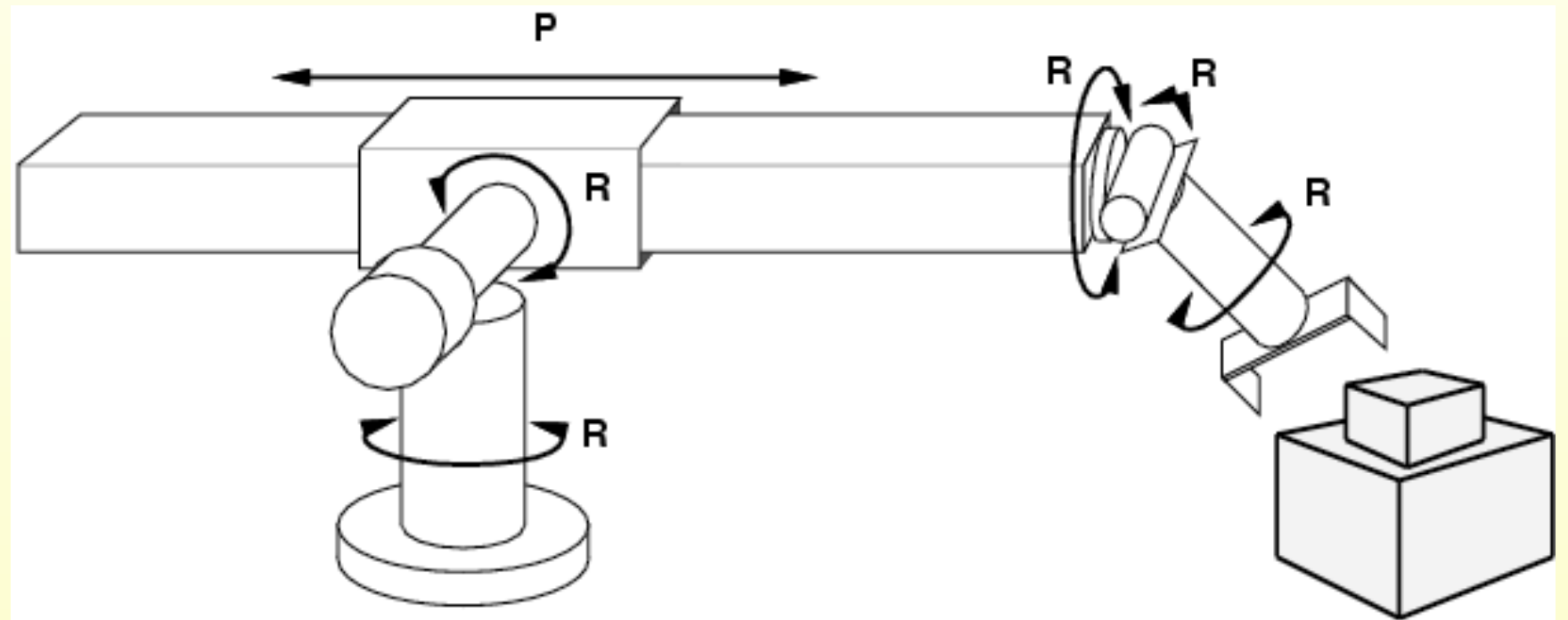
- ❖ continuous motions of robot joints

- ❖ **goal test:**

- ❖ complete assembly

- ❖ **path cost:**

- ❖ time to execute



Uninformed Search

General Tree Search Algorithm

1. Initialize fringe (or frontier) with node containing the initial state

- **Fringe:** data structure with nodes that we want to explore next
- Initially contains a node corresponding to the start state

2. Repeat while you have computational budget:

A. Check if fringe is empty

- If so, return failure

B. (If not) Select a node **n** in the fringe and remove it from the fringe

C. Check if **n** contains a goal state

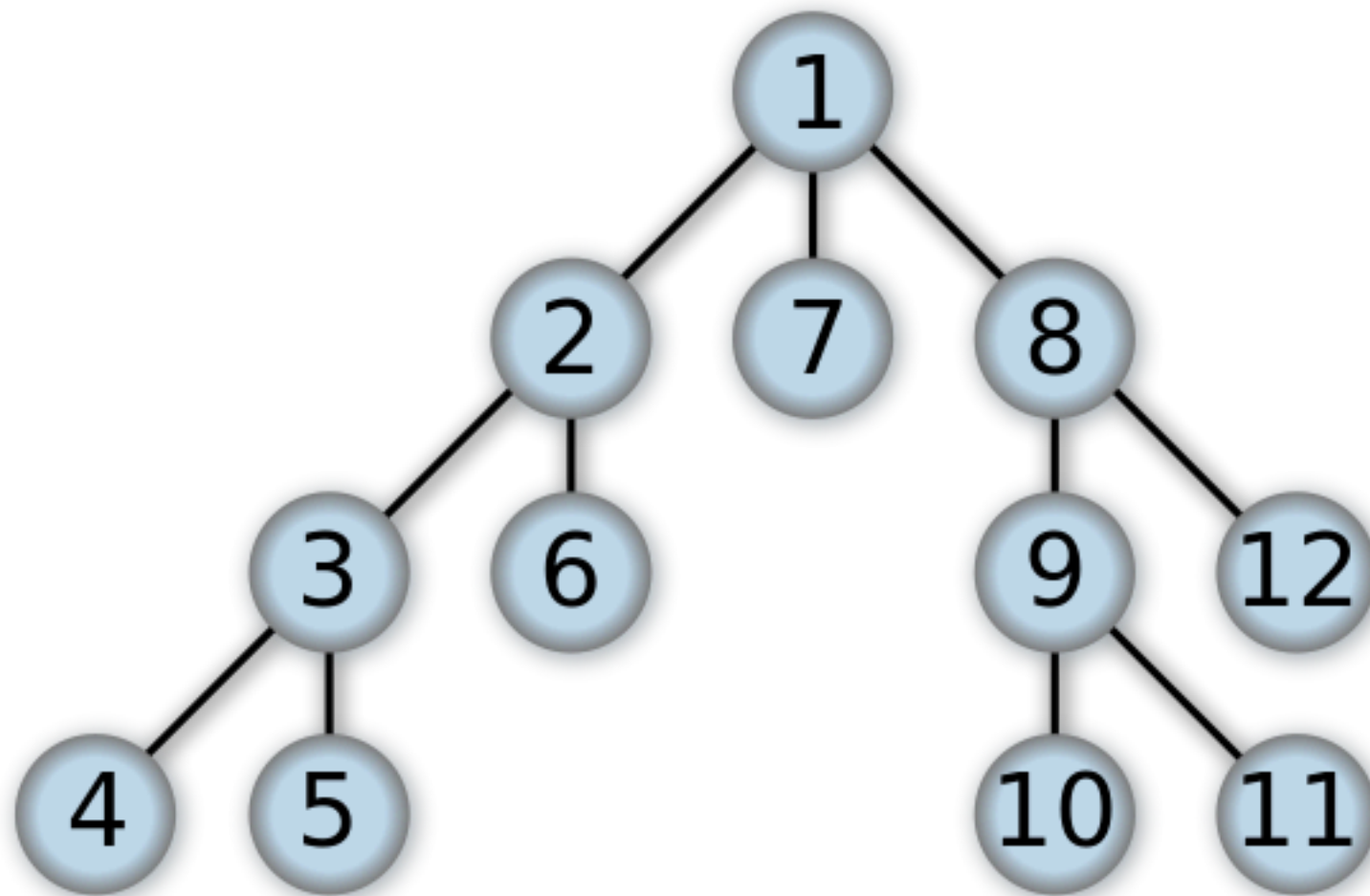
- If so, go to 3 (return solution)

D. (If not) Expand **n**, adding nodes containing all its successor states to the fringe

3. Return the action that leads from the initial node to the second node on the path to the goal (online problems) or the entire path (offline)

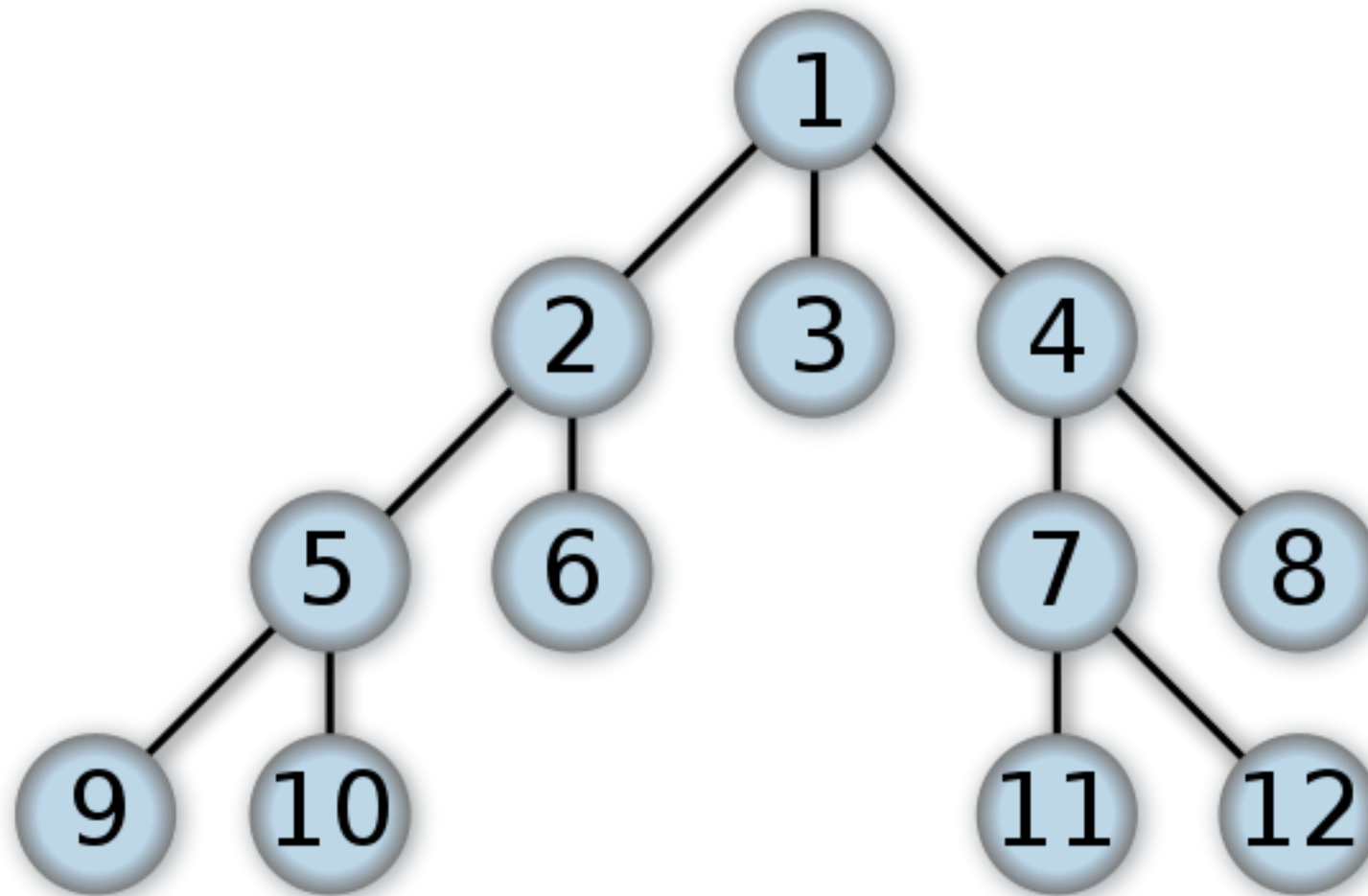
The order in which elements are added (or removed) to the fringe defines the search algorithm!

Depth-first search



- Search deeply first
- Fringe is a LIFO stack

Breadth-first search



- Search widely first
- Fringe is a FIFO queue

DFS versus BFS

- **Depth-First Search (DFS)** goes off into one branch until it reaches a leaf node, then backtracks
 - Low memory usage: Only needs to keep nodes in current path in memory
 - Search is incomplete: may fail to return a solution, even if one exists (loops or infinite branches).
 - Not optimal: if a solution is found, it may not be the cheapest one.

DFS versus BFS

- **Breadth-First Search (DFS)** visits all nodes at a given depth before searching the next level
 - High memory usage: Keeps all nodes at a given level in-memory
 - Search is complete: always finds a solution, if one exists at finite depth
 - Optimal: if a solution is found, it will be the cheapest one (assuming equal edge weights).

DFS versus BFS

- **Breadth-First Search (BFS)** visits all nodes at a given depth before searching the next level
 - High memory usage: Keeps all nodes at a given level in-memory
 - Search is complete: always finds a solution, if one exists at finite depth
 - Optimal: if a solution is found, it will be the cheapest one (assuming cost is uniform)

How can we get the advantages of DFS and BFS at the same time?

Depth-Limited search and iterative Deepening

- Depth-limited: cuts off DFS at a specified depth
- Iterative deepening: iteratively applies depth-limited with higher and higher depths
 - Shallow nodes may be re-visited many times
 - Seems like a lot of overhead at a glance, but asymptotical analysis shows this is not the case (almost all nodes are deep)
 - Combines **low memory usage** of DFS with **completeness and optimality** of BFS. Very good performance in practice!

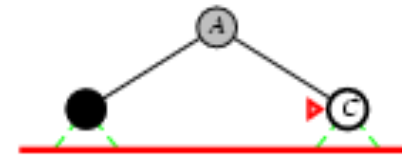
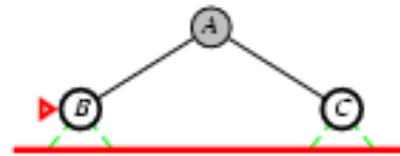
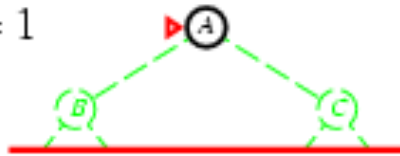
Iterative deepening search $l = 0$

Limit = 0



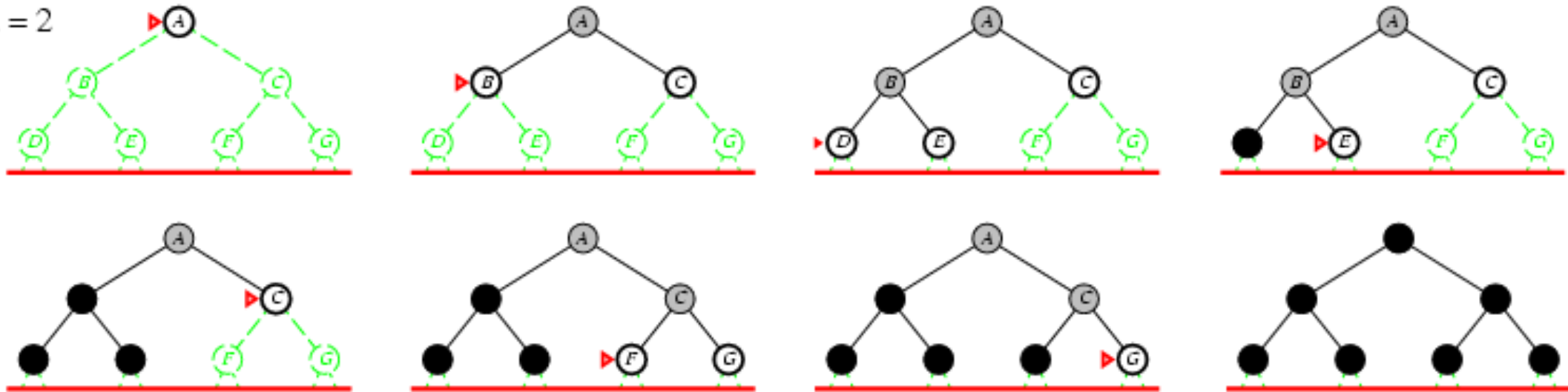
Iterative deepening search $l = 1$

Limit = 1

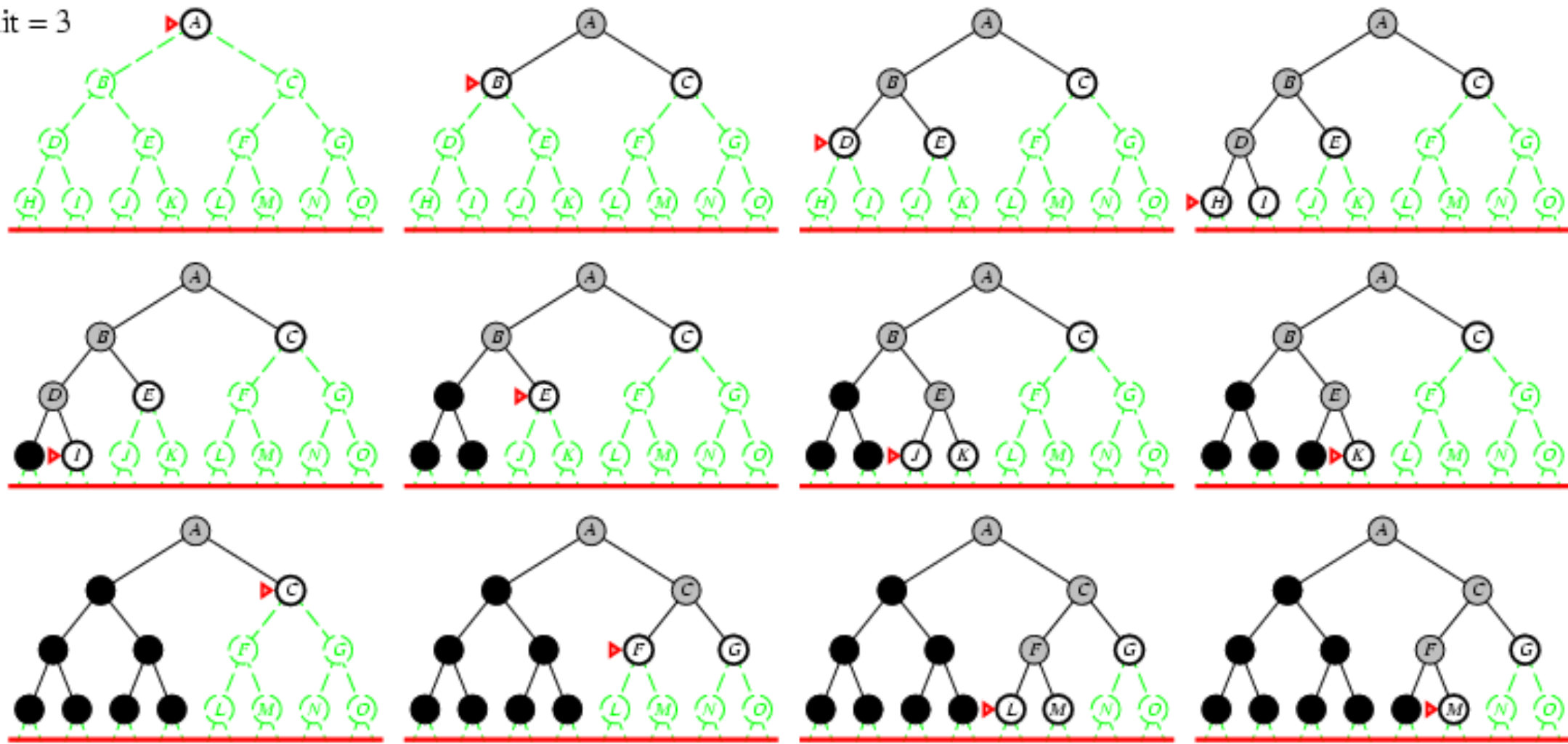


Iterative deepening search $l = 2$

Limit = 2



Iterative deepening search l = 3



Iterative deepening search

- ❖ Number of nodes generated in a depth-limited search to depth d with branching factor b :

$$N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{d-2} + b^{d-1} + b^d$$

- ❖ Number of nodes generated in an iterative deepening search to depth d with branching factor b :

$$N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

- ❖ For $b = 10$, $d = 5$,

- ❖ $N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$

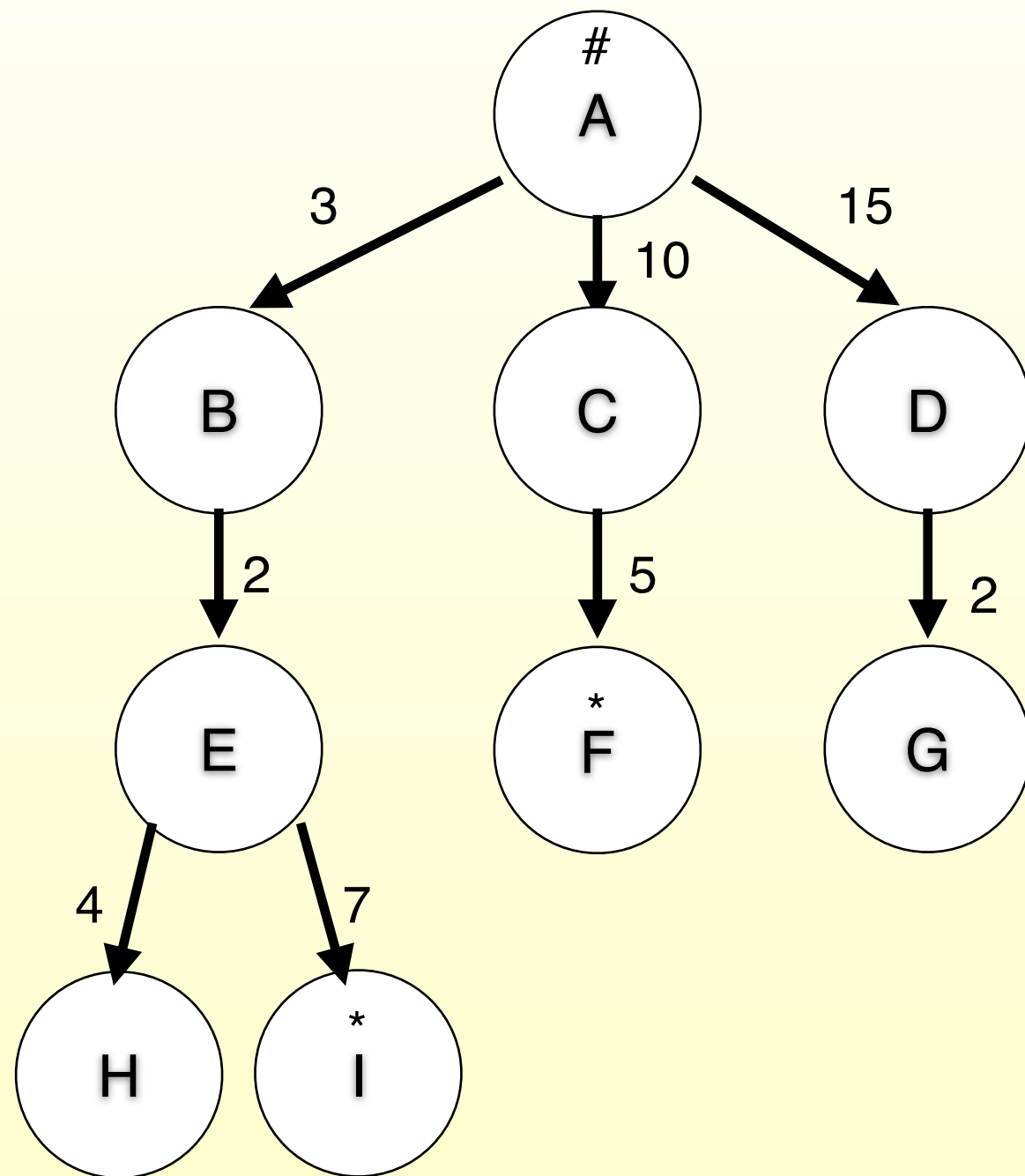
- ❖ $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$

- ❖ Overhead = $(123,456 - 111,111)/111,111 = 11\%$

Uniform Cost search

- A variation of Breadth-First Search that handles edges with variable cost
- Strategy: associate a cost **$g(n)$** to each node n added to the fringe
- Always take the node with smallest g to expand next
- Done naively, this may require sorting the fringe each time, which is expensive.
- **Priority Queues** are an abstract data type that allow for quickly finding the smallest element.
 - Heaps are the most common implementation of priority queues

Example State Space



Legend:

- Starting state

* - Goal state

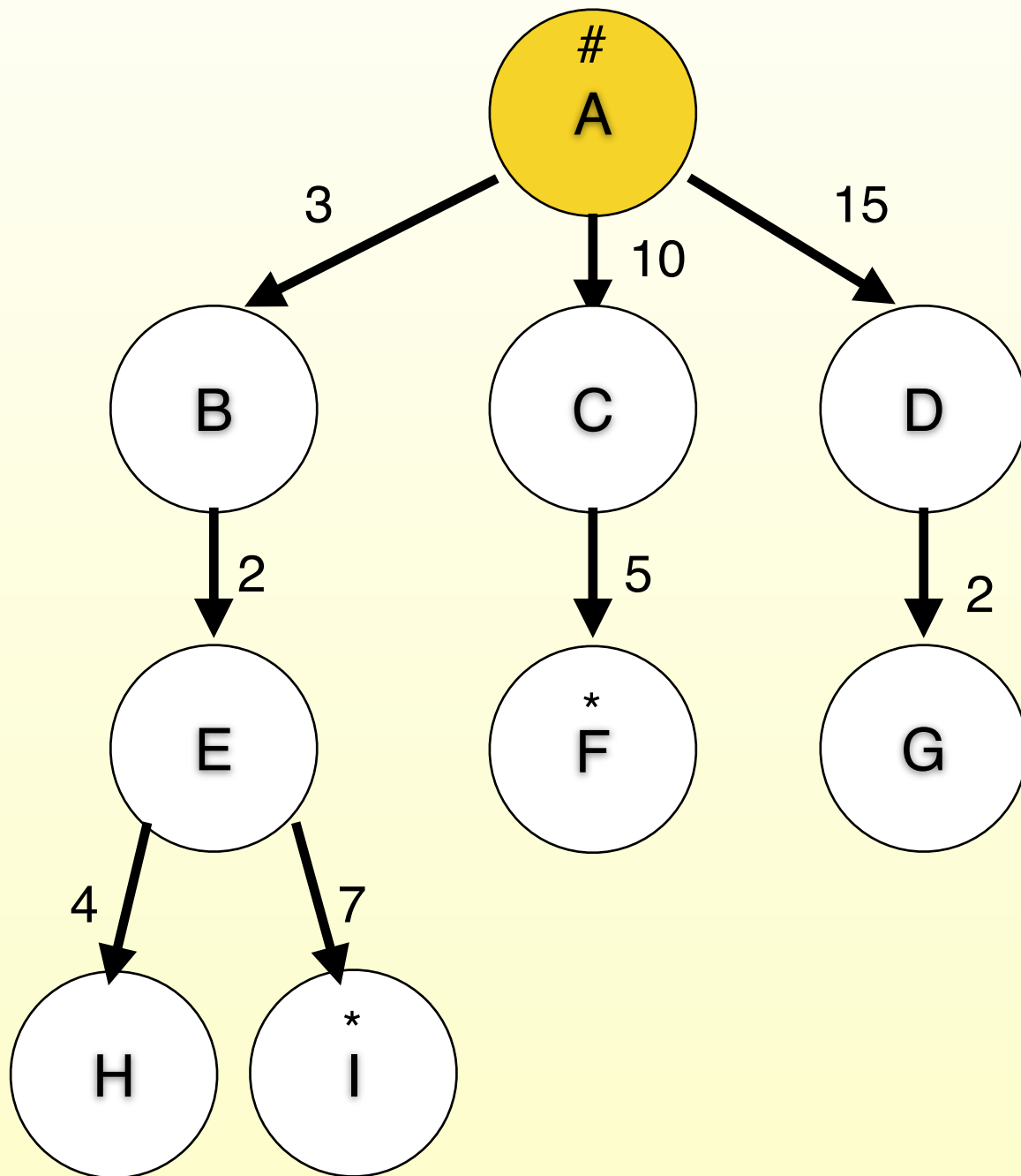
● Visited (expanded)

● Current state

● Fringe

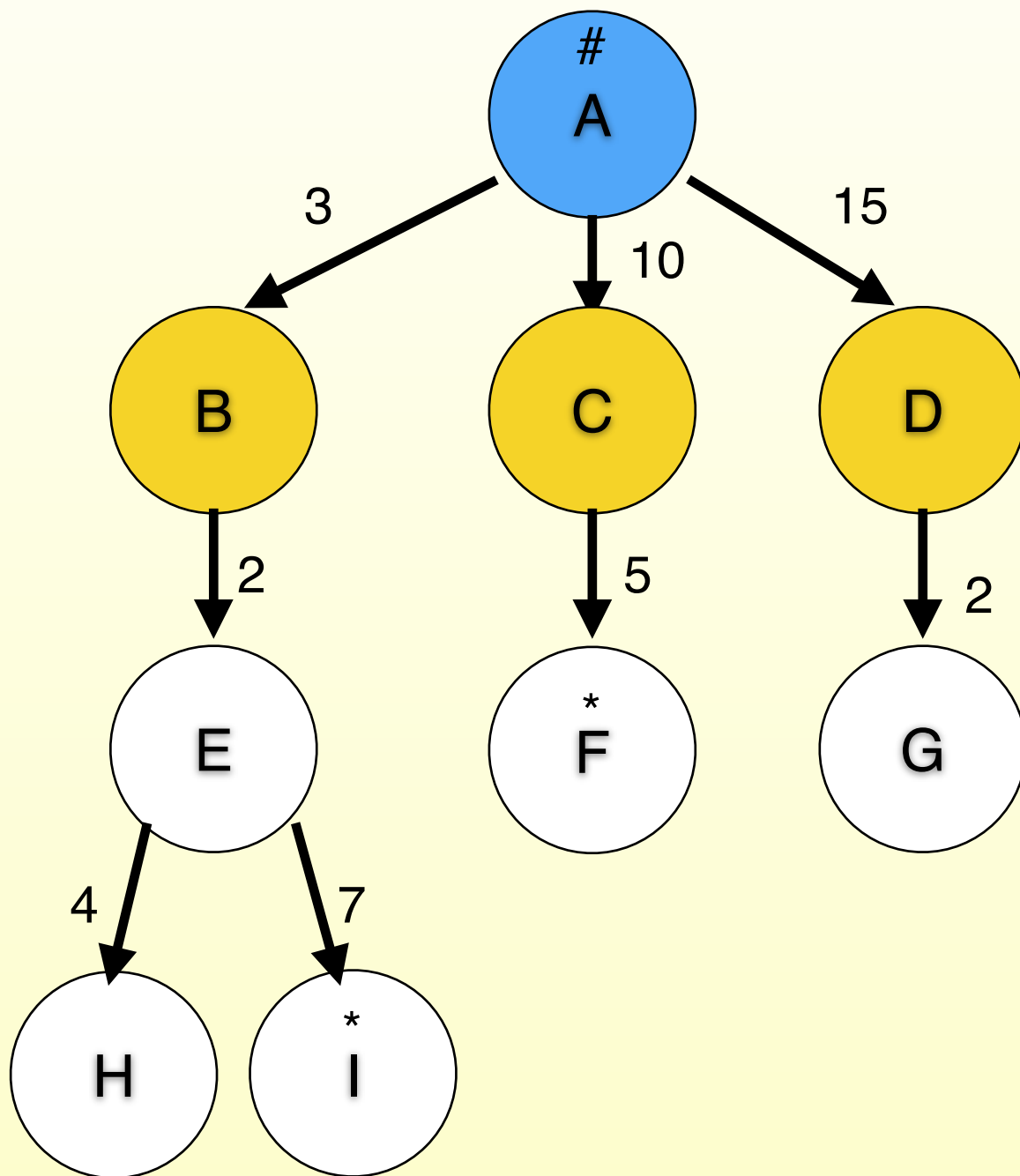
$\xrightarrow{c}$ Transition cost = c

Uniform Cost Search Example



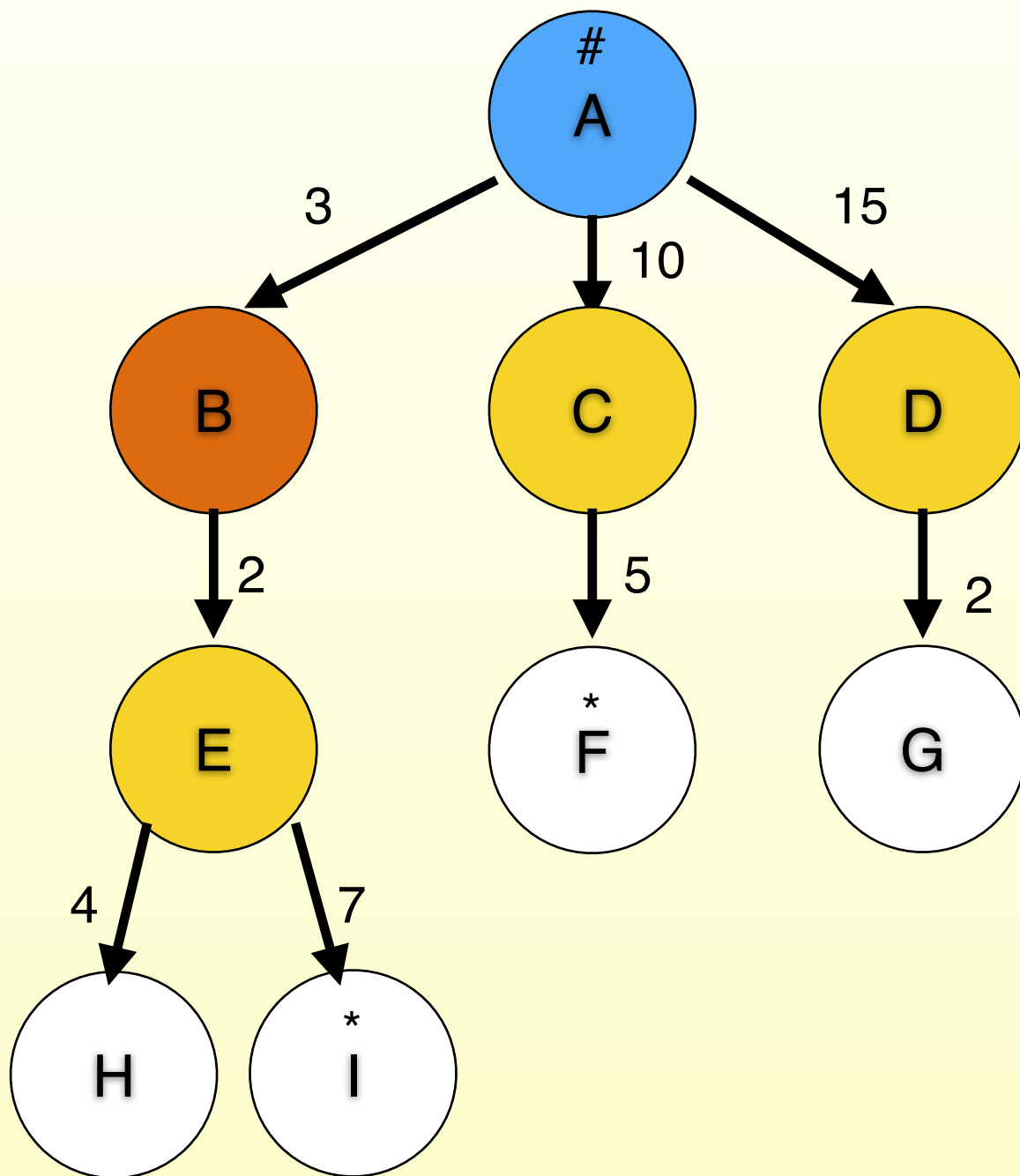
Node	g
A	0

Uniform Cost Search Example



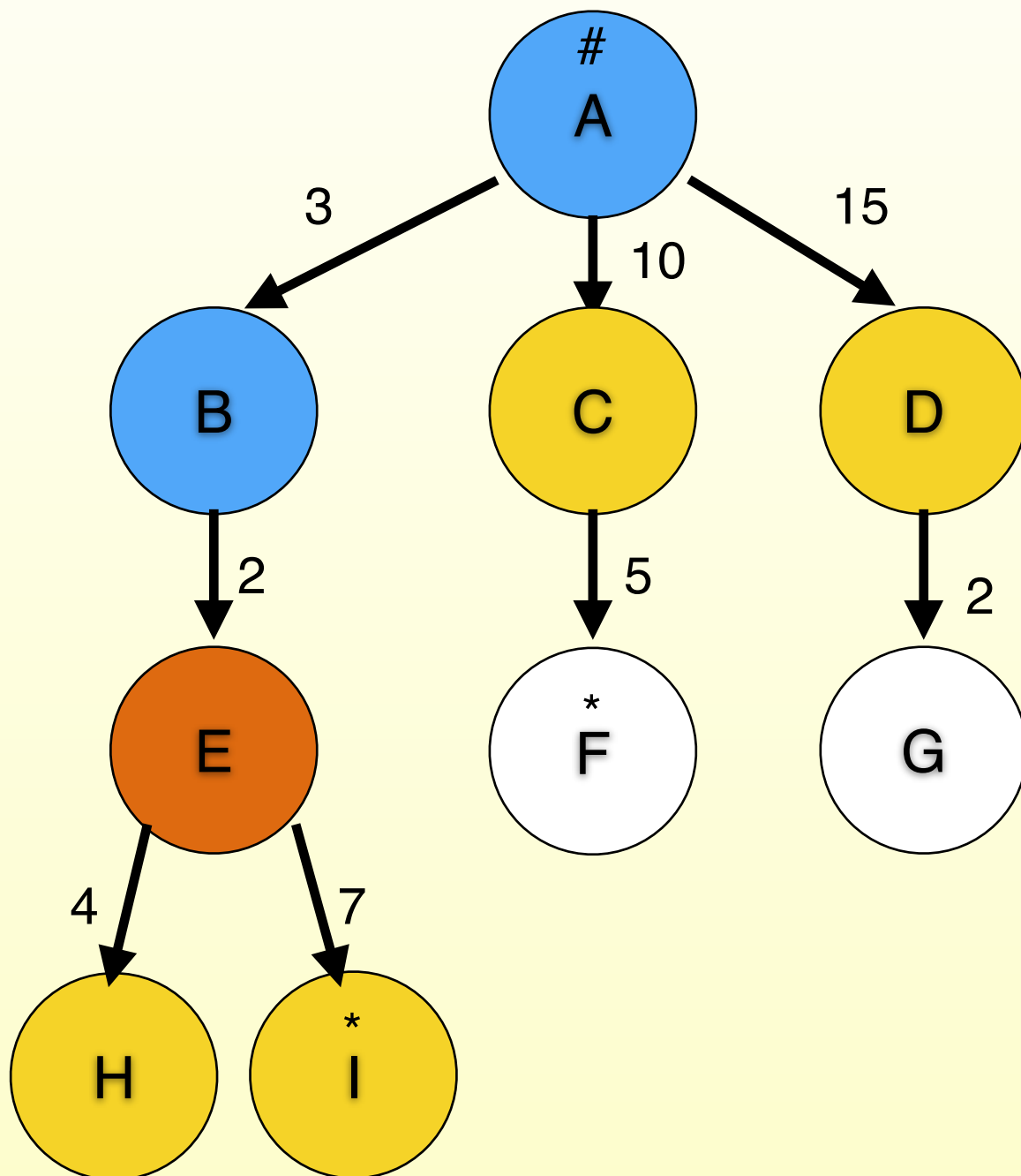
Node	g
B	3
C	10
D	15

Uniform Cost Search Example



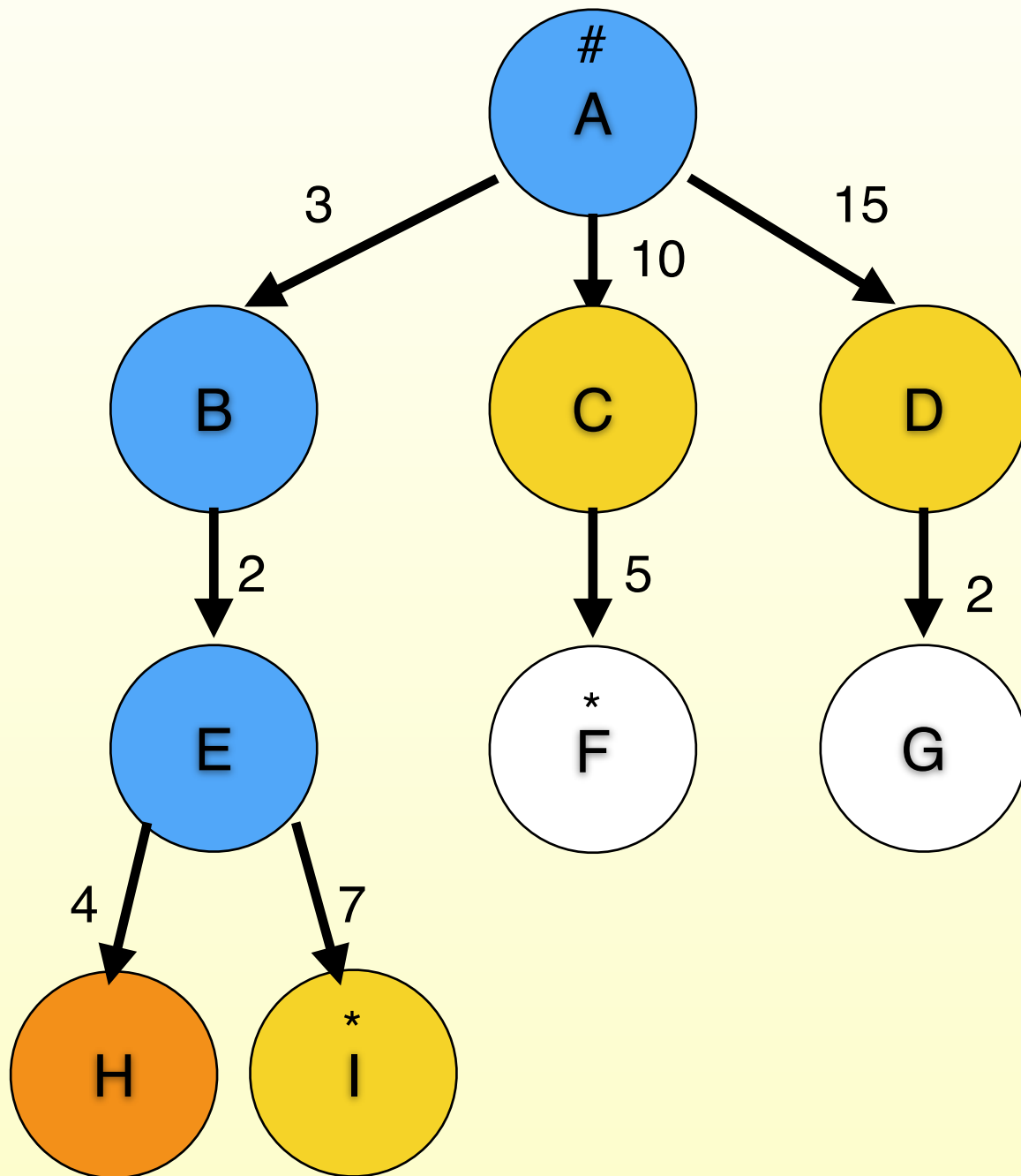
Node	g
E	$3+2 = 5$
C	10
D	15

Uniform Cost Search Example



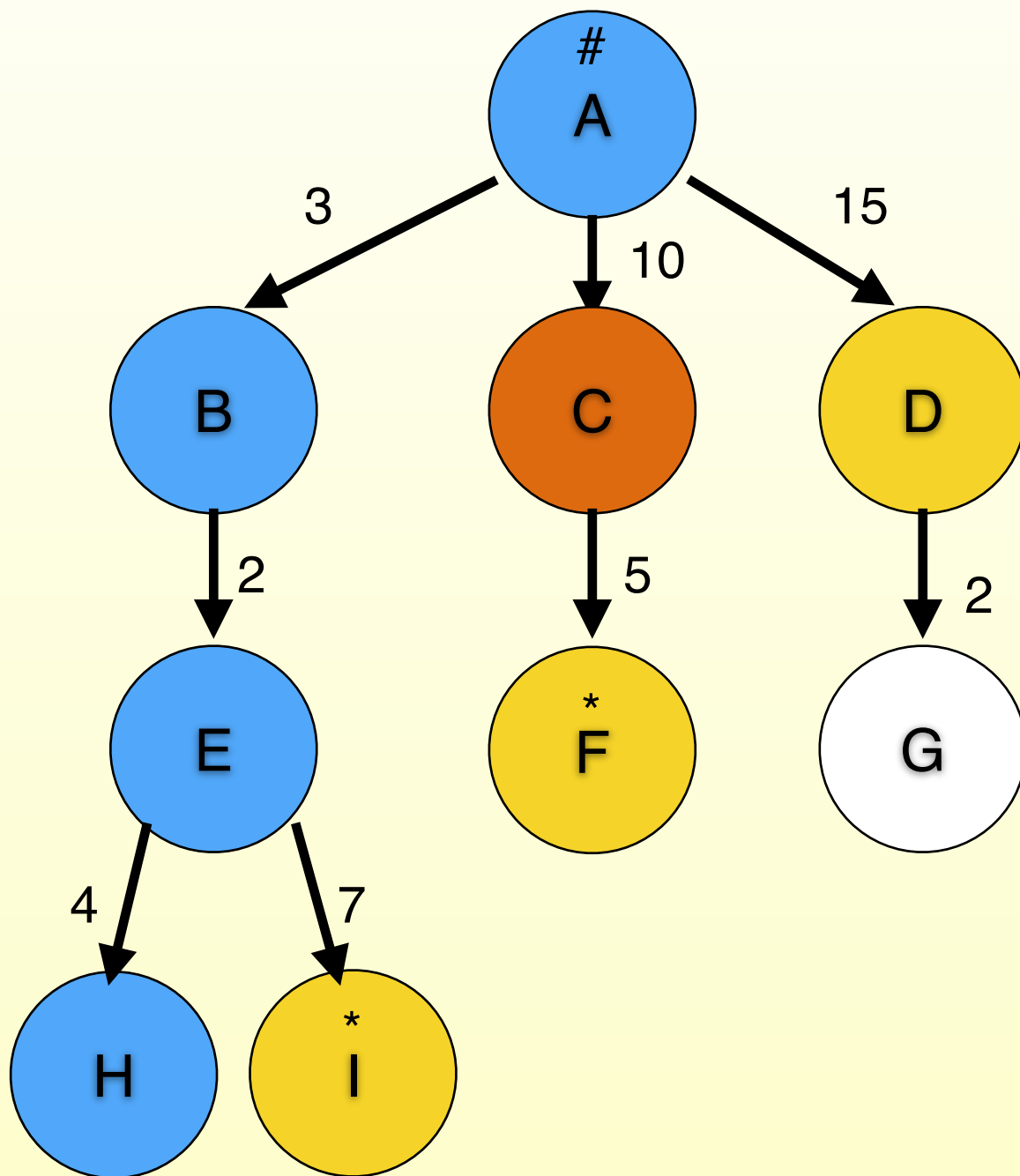
Node	g
H	$5+4 = 9$
C	10
I	$5+7=12$
D	15

Uniform Cost Search Example



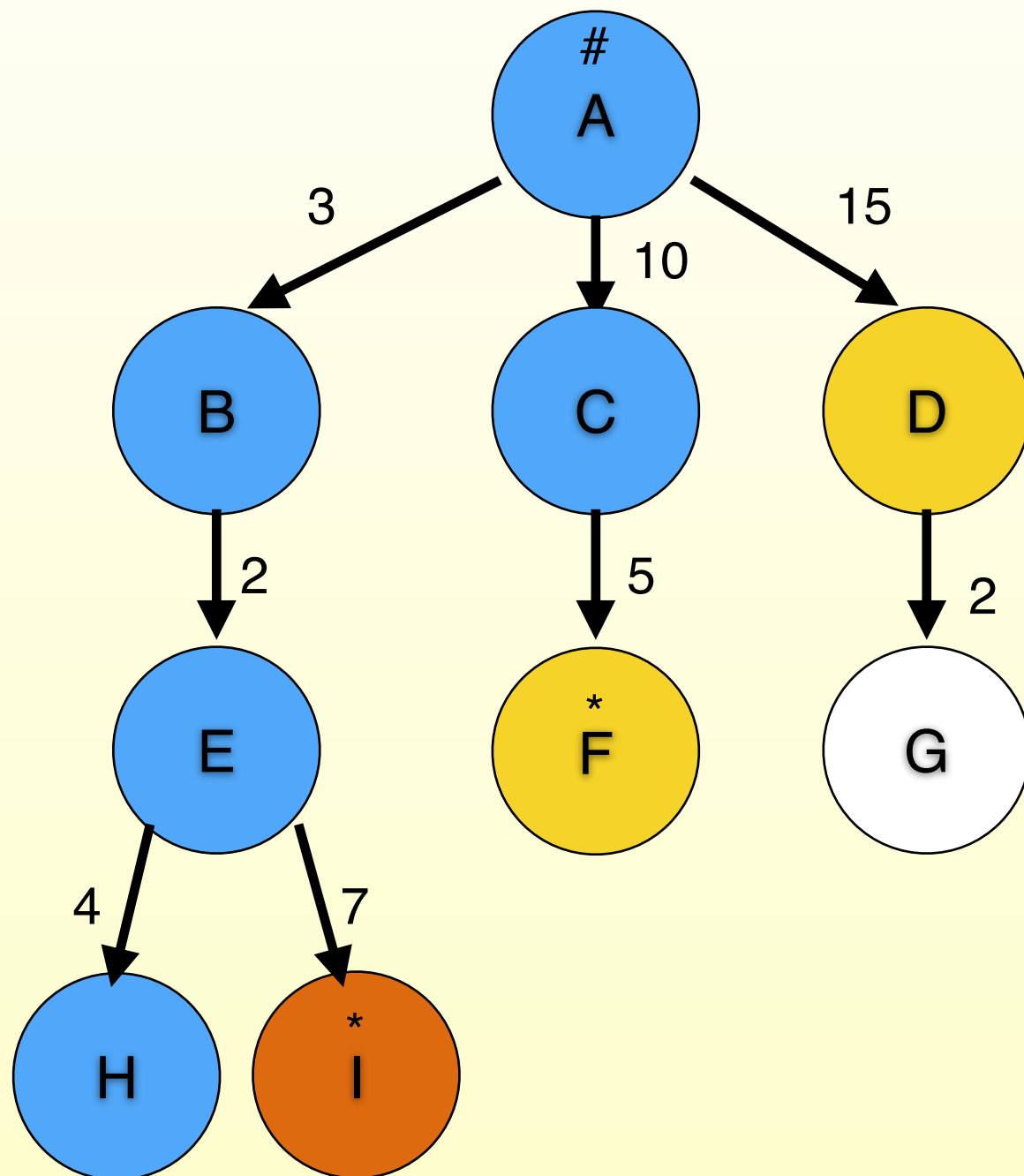
Node	g
H	$5+4 = 9$
C	10
I	$5+7=12$
D	15

Uniform Cost Search Example



Node	g
I	12
F	$10+5=15$
D	15

Uniform Cost Search Example



Goal found with cost 12!

Node	g
I	12
F	$10+5=15$
D	15

Uniform Cost search vs other algorithms

- Uniform cost search can be seen as a variation of Dijkstra's Algorithm
- The typical formulation of Dijkstra is to find the shortest path from the root to **all nodes**
- In uniform cost, we typically stop once the first **goal node** is found
- DFS can be seen as a special case of uniform cost (where all edges have weight 1)
- Uniform cost can be seen as a special case of A^* (where $h=0$ for all nodes)
- See more at https://en.wikipedia.org/wiki/Dijkstra's_algorithm#Related_problems_and_algorithms

Bidirectional search

- Requirements
 - Goal state is unique and known ahead of time
 - States are easily checked for equality (hashable)
 - Transition function must be easily invertible
- Search is conducted from both ends in breadth-first style
- Prototypical use case: maze with 1 entrance and 1 exit
- Advantage
 - More efficient (cost is $\sim$ square root of the cost of regular search)

Summary - Uninformed Search

Method	Complete?	Optimal?	Space complexity	Time complexity	Data structure for fringe	Notes
BFS	Yes (finite b)	Yes (if costs are constant)	exponential	exponential	FIFO (queue)	Complete, Optimal, bad memory complexity
Uniform-Cost	Yes	Yes (even if variable costs)	exponential	exponential	Priority Queue	Handles different costs
DFS	No	No	linear $\sim b \cdot m$	exponential	LIFO (stack)	Low memory requirement, but fewer guarantees than BFS
Backtracking DFS	No	No	linear $\sim m$	exponential	Only 1 node in fringe (but may use stack as part of backtracking implementation)	Even lower memory requirement, but a bit more complicated than DFS
Depth-Limited	No	No	Linear $\sim b \cdot l$	exponential	Stack (variation of BFS)	Alleviates DFS problems with infinite spaces and loops
Iterative Deepening	Yes (if reachable with available memory)	Yes (if reachable with available memory)	Linear $\sim b \cdot d$	exponential	Stack (variation of BFS)	Combines low memory of DFS with completeness and optimality of BFS. Re-explores shallow nodes many times (not too bad)
Bi-directional	Yes (if using BFS)	Yes (if using BFS)	Exponential (square root of complexity with BFS)	exponential (square root of complexity with most other strategies)	Depends on which strategy each side is using	Needs to generate predecessors, needs to enumerate goals, at least one side needs to store all frontier nodes

Note: b = branching factor, d = depth of shallowest goal, m = largest depth of any path, l = threshold for depth-limited

